

Progress Report: Agentic Python Verifier

Harry Wang

Eric Shang Nan Chen

May 6, 2026

1 Project Overview

The project is a proof-driven code generation pipeline for Python. Instead of generating Python first and checking it afterward, the system first asks a language-model synthesis agent to produce a PlusCal algorithm, a candidate inductive invariant, and a target safety property from a natural-language requirement. The formal verifier then translates the PlusCal module to TLA+, checks three proof obligations with TLC, and returns structured counterexamples to the synthesis agent when a proof fails. Only after all three obligations pass does a refinement agent translate the verified PlusCal artifact into executable Python with runtime assertions derived from the verified invariant.

The current repository contains a working first pipeline implementation, command-line interface, formal checking layer, language-model tool contracts, offline tests, integration-test scaffolding, and two example benchmark requirements.

2 Implemented Pipeline

The implemented pipeline is centered in `src/main.py`. It accepts a `TaskRequest`, loads runtime settings, creates output directories, and then runs the following loop:

1. The synthesis agent proposes a tuple (`PlusCal`, `Inv`, `Property`, `finite constants`).
2. The verifier checks initiation, consecution, and property implication.
3. If any obligation fails, the failing TLC result is serialized and sent back to the repair agent through Anthropic tool-result history.
4. If all obligations pass, the refinement agent emits a Python module.
5. Verified artifacts are written to `generated/tla` and `generated/python`, with per-iteration scratch files in `generated/work`.

The pipeline returns a structured `PipelineResult` with either `verified` or `unverified` status. Verification failures do not silently pass through to Python generation; Python is emitted only when the full proof bundle passes.

Obligation	Mathematical Form	TLC Encoding
Initiation	$\text{Init} \Rightarrow \text{Inv}$	<code>SPECIFICATION Spec, INVARIANT Inv</code>
Consecution	$\text{Inv} / \text{Next} \Rightarrow \text{Inv}'$	<code>Auxiliary IInit == Inv, ISpec == IInit / [] [Next]_vars</code>
Property implication	$\text{Inv} \Rightarrow \text{Property}$	<code>Auxiliary PInit == Inv, unchanged-step PSpec</code>

Table 1: Implemented proof obligations.

3 Formal Verification Progress

The verifier in `src/agents/verifier.py` implements three model-checking obligations for a proposal with initial predicate `Init`, transition relation `Next`, inductive invariant `Inv`, and target property `Property`.

The `src/formal/tlc_config.py` module generates the TLC configuration files and the two auxiliary TLA+ modules used for consecution and property checking. The property check enumerates all invariant states by using `PInit == Inv` and an unchanged transition relation, then asks TLC to check `Property` as an invariant over those states.

The implementation also wraps the two required TLA+ tools:

- `src/formal/pcal_translator.py` invokes `pcal.trans` from `tla2tools.jar` and checks that a translation block was inserted.
- `src/formal/tla_runner.py` invokes `tlc2.TLC` with a configurable timeout, automatic workers, and deadlock checking enabled.

4 Counterexample Feedback

Structured counterexample feedback is implemented in `src/formal/counterexample_parser.py`. The parser classifies TLC runs as `passed`, `failed`, `timeout`, or `error`. For invariant and initial-state violations it extracts:

- the failing obligation, such as `consec`;
- the violated predicate, such as `Inv`;
- each trace state index and action;
- variable assignments from TLC trace lines.

This information is then serialized by `SynthesisAgent.repair`. For example, a consecution failure may be sent back to the model as:

```
- consec: FAILED
  violated_predicate: Inv
  State 1 <Initial predicate>: counter=11
```

This is a major milestone because the system is not limited to one-shot generation. It has a repair loop with formal counterexamples as the feedback signal.

5 Language-Model Agent Progress

The language-model boundary is implemented with explicit Anthropic tool schemas in `src/llm/tools.py`. Three tools are currently defined:

- `propose_pluscal_with_invariant`: emits the initial PlusCal module, invariant, property, and finite constant domains.
- `repair_after_counterexample`: emits a revised proposal after a failed proof obligation.
- `emit_python_module`: emits Python source and an assertion map from invariant clauses to Python checks.

The prompt layer in `src/llm/prompts.py` gives the synthesis model specific formal-methods constraints. In particular, it requires small finite constant domains, explicit variable membership in `Inv`, and explicit enumeration of the PlusCal `pc` variable. These details are important because the consecution and property checks use `Inv` as an initial state predicate, so TLC must be able to enumerate states from the invariant alone.

The Anthropic client wrapper in `src/llm/anthropic_client.py` supports prompt caching through ephemeral cache-control system blocks and retries once on overloaded-model errors using a fallback model.

6 Python Refinement Progress

The refinement stage is implemented in `src/agents/refine_agent.py`. It extracts top-level conjuncts from the verified invariant and passes them to the refinement model. The emitted Python module is required to include runtime assertions at the entry and exit of state-mutating functions and to provide an `assertion_map` for traceability.

For example, given this invariant:

```
Inv == counter >= 0
      /\ counter <= 10
      /\ counter \in 0..10
```

the refinement prompt sends the model the individual conjuncts:

```
- counter >= 0
- counter <= 10
- counter \in 0..10
```

The intended Python result is an implementation that carries corresponding assertions, such as:

```
def increment(counter: int) -> int:
    assert counter >= 0
    assert counter <= 10
    if counter < 10:
        counter += 1
    assert counter >= 0
    assert counter <= 10
    return counter
```

7 Examples

Two benchmark prompts have been added in the `examples` directory:

```
Implement a bounded counter from 0 to 10.
```

```
Implement a bank transfer between two accounts that never allows a negative  
balance and preserves the total sum of money.
```

The test fixtures include a good bounded-counter TLA+ module and a deliberately bad invariant module. The good fixture uses:

```
Inv == /\ counter \in 0..MaxValue  
      /\ pc \in {"Increment", "Done"}  
  
Property == counter <= MaxValue
```

This shows the expected invariant style: the counter is bounded by explicit set membership and the PlusCal control state is enumerated. The bad fixture uses `counter \in 0..(MaxValue-1)`, which is too strong after the final increment. The integration test expects this module to fail an initiation or consecution obligation with a counterexample.

8 Command-Line Interface

A Typer-based CLI is implemented in `src/cli.py`. It exposes:

- `--output` for generated artifacts;
- `--max-iterations` with bounds from 1 to 20;
- `--model` for overriding the Anthropic model;
- `--verbose` for debug logging.

The CLI uses distinct exit codes: 0 for verified output, 1 for exhausted repair iterations, and 2 for missing runtime configuration such as `ANTHROPIC_API_KEY` or `TLA2TOOLS_JAR`.

9 Testing Progress

The repository includes focused unit tests and mocked pipeline tests:

- model validation tests for task requests, synthesis proposals, constants, repair proposals, and proof bundles;
- configuration tests for missing API keys and TLA+ tools;
- CLI smoke tests for help text and fail-fast configuration errors;
- counterexample parser tests using captured TLC output fixtures;
- TLC config generation tests for initiation, consecution, and property configurations;
- mocked synthesis-agent tests for tool-call extraction and repair feedback serialization;

- mocked refinement-agent tests for invariant-conjunct extraction;
- mocked end-to-end pipeline tests covering first-try success, repair then success, and exhausted iterations;
- gated integration tests for running real TLC against known good and known bad bounded-counter fixtures.

The offline test suite is designed to run without an Anthropic API key or `tla2tools.jar`; the external dependencies are mocked at the boundary. The TLC integration tests are explicitly marked and skipped unless a valid TLA+ tools jar is configured.

10 Current Limitations

The prototype currently targets finite-domain inductive checking with TLC. Unbounded inductive proofs would require a theorem prover such as TLAPS, which is outside the current scope. The invariant must be enumerable as an initial state predicate, which means every state variable needs an explicit finite membership constraint. The refinement step depends on the language model to translate verified PlusCal into faithful Python, so the runtime assertions provide an additional guard but do not replace independent Python-level tests.

There is also inherent nondeterminism in live LLM calls. The current design mitigates this by making the verified TLA+ module and proof results the trusted intermediate artifact, while the repository tests mock the Anthropic boundary for repeatability.

11 Next Steps

The next development milestones are:

1. Run live end-to-end examples with a real Anthropic API key and `tla2tools.jar`, then save representative generated artifacts.
2. Add Python-level tests for refined modules so generated runtime assertions are exercised after formal verification succeeds.
3. Expand the benchmark set beyond bounded counters and bank transfers.
4. Improve invariant-conjunct extraction so it handles nested TLA+ expressions more robustly than the current regular-expression split.
5. Add reporting or logs that summarize each repair iteration and the obligation that drove it.

12 Summary

Substantial progress has been made toward a complete proof-driven Python code generation system. The project now has a working architecture for PlusCal/invariant co-synthesis, three formal TLC proof obligations, structured counterexample-driven repair, verified-artifact output, Python refinement with invariant-derived assertions, a usable CLI, examples, and broad offline test coverage. The remaining work is mainly in live evaluation, broader benchmarks, and hardening the refinement and reporting layers.